



Personal Portfolio Website Project

PREPARED FOR

Software Engineering Course AUT

PREPARED BY

AmirParsa Khadem

Dr. Zahra Ghorbanali

Date

Sep 2025

عنوان پروژه: ساخت پورتفولیوی شخصی

این پروژه شامل طراحی و پیاده‌سازی یک وبسایت شخصی (پورتفولیو) است که به صورت ریسپانسیو با استفاده از **HTML، CSS و JavaScript** پیاده‌سازی شده (بدون فریم ورک) و روی **GitHub Pages** منتشر می‌شود.

برنامه زمان‌بندی و ددلاین‌ها:

برای پیش‌برد بهتر پروژه، آن را به ۳ مرحله تقسیم می‌کنیم. هر مرحله ددلاین خاص خود را دارد:

ددلاین اول: آماده‌سازی GitHub و پروژه‌ها

مهلت: 5 روز

وظایف:

- ایجاد یا بهبود اکانت GitHub
- ویدیوها:
 - <https://youtu.be/lfVVm5K-Xp0?si=Dn7W2c9JQznkzh1q>
 - <https://youtu.be/rCt9DatF63I?si=8SdxkXZbYL2NzFZS>
- اضافه کردن حداقل ۴ پروژه واقعی همراه با:

- توضیح پروژه
- فایل README.md معقول و کامل

- نام‌گذاری مناسب ریپوها و مرتب‌سازی ساختار GitHub

ددلاین دوم: آماده‌سازی CV

مهلت: ۳ روز پس از ددلاین اول

وظایف:

- نوشتن CV حرفه‌ای شامل:
 - اطلاعات تماس (ایمیل، لینکدین، گیت‌هاب و...)
 - اطلاعات تحصیلی
 - تجربه کاری (در صورت وجود)
 - تجربه تدریس‌یار (اگر بوده)
 - مهارت‌ها (فنی/هارد اسکیل‌ها)
 - پروژه‌ها (حداکثر ۴ مورد با لینک گیت‌هاب و توضیح کوتاه)
 - زبان‌ها
- رزومه باید حداکثر ۲ صفحه باشد
- ترجیحا با استفاده از Latex تهیه شود و فرمت PDF

ددلاین سوم: ساخت و انتشار وب‌سایت

مهلت: ۵ روز پس از ددلاین دوم

ساختار وب‌سایت (اجباری):

بخش‌های اصلی:

1. Home

- معرفی کوتاه + تصویر

2. About Me

- اطلاعات شخصی، علاقه‌مندی‌ها، مهارت‌ها
- قابلیت دانلود فایل CV

3. Experience

- تجربه‌های کاری یا آکادمیک

4. Projects

- حداقل ۳ و حداکثر ۷ پروژه
- هر پروژه باید شامل توضیح + لینک GitHub با README معقول

5. Contact

- لینک GitHub، ایمیل، LinkedIn

6. Navigation Bar

- نَوَ بار قابل استفاده در موبایل و دسکتاپ

الزامات فنی:

- فقط از **HTML, CSS, JavaScript** استفاده شود (بدون فریم‌ورک)
- وب‌سایت باید **ریسپانسیو (Responsive)** باشد
- کد در یک ریپوی تمیز روی GitHub قرار گیرد
- پروژه باید روی GitHub Pages منتشر شود
- فایل README.md در ریپو توضیح کامل پروژه را شامل شود

منابع و آموزش‌های لازم برای ساخت و انتشار سایت

HTML، CSS و JavaScript چی هستند و هر کدام چه کاری می‌کنند؟

HTML (HyperText Markup Language)

استخوان‌بندی صفحه‌ی وبه.

با HTML ساختار کلی صفحه رو مشخص می‌کنی:

مثلاً قراره یه عنوان داشته باشی، یه متن، چندتا عکس، فرم تماس، دکمه، و...

مثال:

- این یه تیتره
- این یه پاراگرافه
- این یه تصویره
- این یه فرم ارساله

CSS (Cascading Style Sheets)

ظاهر و طراحی صفحه رو زیبا می‌کنه.

با CSS می‌تونی به المان‌هایی که با HTML ساختی، استایل بدی.

مثلاً رنگشون رو عوض کنی، فونت تعیین کنی، حاشیه بدی، سایه بندازی، یا ریسپانسیوش کنی.

مثال:

- رنگ دکمه سبز باشه
- متن وسط‌چین باشه
- تصویر گرد باشه
- توی موبایل همه چیز بیاد زیر هم

JavaScript (JS)

صفحه رو زنده و تعاملی می‌کنه.

با JS می‌تونی به کاربر واکنش نشون بدی، مثلاً وقتی یه دکمه کلیک شد، چیزی نمایش داده بشه یا فرم بررسی

بشه.

جاوااسکریپت اجازه می‌ده صفحه فقط به چیز ثابت نباشه.

مثال:

- وقتی روی دکمه کلیک شد، به پیام نمایش داده بشه
- محتوای پروژه‌ها به صورت داینامیک لود بشه
- فرم تماس قبل از ارسال بررسی کنه که همه‌ی فیلدها پر شدن یا نه

طراحی ریسپانسیو (Responsive Design) چیه؟

طراحی ریسپانسیو (Responsive Design) یعنی طراحی یک وبسایت به گونه‌ای که در هر نوع دستگاهی (موبایل، تبلت، لپ‌تاپ، دسکتاپ و...) ظاهر مناسب، خوانا و کاربرپسند داشته باشه.

هدفش چیه؟

این‌که کاربر نیازی نداشته باشه توی صفحه اسکرول افقی کنه، زوم کنه یا دکمه‌ها رو سخت پیدا کنه. وبسایت باید خودش با توجه به سایز صفحه نمایش، خودش رو تطبیق بده.

چرا مهمه؟

چون بیشتر کاربران امروز از موبایل استفاده می‌کنن. اگه وبسایت فقط برای دسکتاپ طراحی شده باشه، تجربه کاربری بدی روی موبایل خواهد داشت و این یعنی:

- کاربر سریع وبسایت رو ترک می‌کنه
- احتمال تعاملش با سایت پایین میاد

🌐 GitHub Pages چیه؟

GitHub Pages به سرویس رایگان از سایت GitHub هست که بهت اجازه می‌ده کدهای HTML/CSS/JS خودت رو به راحتی تبدیل به یه وبسایت واقعی آنلاین کنی و بدون نیاز به هاست یا سرور، اون رو منتشر کنی.

با GitHub Pages می‌تونی:

- یه وبسایت شخصی یا پورتفولیو داشته باشی

- سایت مستندات برای پروژه‌هاست بسازی
- رزومه یا نمونه کارها رو به صورت آنلاین نشون بدی

آموزش سریع راه‌اندازی:

برای اینکه پروژه‌ت رو روی GitHub Pages منتشر کنی، می‌تونی قدم به قدم با این ویدیو پیش بری:

 [آموزش انتشار وبسایت روی YouTube \(GitHub Pages\)](#)

با دیدن این ویدیو می‌تونی در چند دقیقه پروژه‌ت رو روی گیت‌هاب پیجز بیاری بالا و یه لینک عمومی داشته باشی که هر کسی بتونه ببینه.

منابع کمکی برای انجام پروژه

برای اینکه راحت‌تر بتونید پروژه‌ی پورتفولیو شخصی‌تون رو پیش ببرید، از ویدیوهای آموزشی زیر استفاده کنید. این ویدیوها مراحل ساخت یک وبسایت ساده با HTML، CSS و JavaScript رو آموزش می‌دن و بهتون کمک می‌کنن ساختار کلی و نحوه‌ی پیاده‌سازی رو بهتر درک کنید:

[ویدیو اول: Build a Personal Portfolio Website Using HTML & CSS](#) 

[ویدیو دوم: Responsive Personal Portfolio Website Design](#) 

پیشنهاد مهم: قدم به قدم با ویدیو جلو برید

- هر قسمت از ویدیو رو ببینید، بعد همون بخش رو خودتون پیاده‌سازی کنید
- هر بار که یک تغییر منطقی در پروژه دادید (مثلاً: ساخت نوبار، اضافه کردن سکشن "درباره من"، طراحی ریسپانسیو پروژه‌ها و...) حتماً:

○ یک کامیت (commit) معنادار و واضح توی گیت بزنید

این کار هم پروژه‌تون رو قابل ردیابی می‌کنه، هم نشون می‌ده مرحله به مرحله با فکر جلو رفتید (نه اینکه همه چیز رو یکباره بریزید).

اگر نمی‌دونی گیت و گیت‌هاب چیه یا تا حالا استفاده نکردی، این ویدیوها بهت کمک می‌کنن شروع کنی و اصول کار با این ابزارها رو یاد بگیری.

گیت (Git) یک سیستم کنترل نسخه است که بهت اجازه می‌ده تغییرات کدت رو مرحله‌به‌مرحله ذخیره کنی، راحت‌تر با دیگران همکاری کنی و هر وقت خواستی به نسخه‌های قبلی برگردی.

آموزش گیت انگلیسی:

<https://youtu.be/8JJ101D3knE?si=KRb-H49iQsIRi3g> 

آموزش گیت فارسی:

<https://youtu.be/Ac2FaOuPF2Q?si=bfYwsYB2lpUiurXi> 

گیت چطوری کار می‌کنه؟

https://youtu.be/e9lmsKot_SQ?si=XBVUPNmc_ZI_S6cM 

Commit Policy

برای تمیز و حرفه‌ای بودن تاریخچه‌ی گیت، لطفاً هنگام کامیت کردن این اصول را رعایت کنید.

1. ساختار پیام کامیت

هر پیام باید این الگو را دنبال کند:

```
<type>: <short summary>

[optional body: why/how]
[optional footer: issue references, notes]
```

2. نوع (type) کامیت

- **init** → ایجاد اسکلت اولیه‌ی پروژه
- **feat** → اضافه کردن یک قابلیت جدید (مثلاً Navbar، فرم تماس)
- **fix** → رفع باگ یا مشکل
- **style** → تغییرات ظاهری (CSS، رنگ، فاصله‌ها) بدون تغییر منطق
- **refactor** → تغییر ساختار یا پاک‌سازی کد بدون افزودن قابلیت جدید
- **docs** → تغییرات در مستندات (README و ...)
- **chore** → تغییرات متفرقه (تنظیمات، ignore فایل و ...)

3. اصول نوشتن پیام

- خلاصه (summary) حداکثر ۷۲ کاراکتر باشد.
- از انگلیسی ساده و فعل امری استفاده کنید:
- **feat: add responsive navbar** ✓
 - **من نویار رو درست کردم** ✗
- در صورت نیاز، توضیحات کامل‌تر را در بخش **body** اضافه کنید.

4. فرکانس کامیت

- هر تغییر منطقی و مستقل → یک کامیت جدا

- ❌ همه چیز در یک کامیت بزرگ
- ✅ هر بخش یا فیچر در یک کامیت مشخص

5. نمونه‌های خوب

- `init: scaffold project structure with HTML/CSS/JS`
- `feat: add Home and About Me sections`
- `feat: implement responsive navbar with toggle`
- `style: adjust typography and spacing`
- `feat: add project cards with GitHub links`
- `docs: update README with project description and screenshots`
- `fix: correct LinkedIn link in contact section`

این فرمت باعث می‌شود:

- تاریخچه پروژه تمیز و قابل فهم باشد
- مرور تغییرات در آینده راحت تر شود
- حرفه‌ای بودن ریپو

هدف و ددلاین پروژه

هدف از این پروژه این است که مهارت شما در کار با Git سنجیده شود و همچنین با GitHub Pages آشنا شوید.

اگر تا امروز با Git کار نکردید، فرصت خوبی است که با انجام این پروژه یادگیری را به صورت عملی شروع کنید (**learn by doing**).

ددلاین نهایی روز 11 مهر است و به هیچ وجه تمدید نخواهد شد.